RFC: Organization and Definition of Syntax for Layered Haskell

Status: Proposed
Project: Layered Haskell
Decision: How to organize and formally define the syntax of each language layer

1. Problem and Context

Our Layered Haskell project aims to make a simpler version of Haskell for teaching students about functional programming. Rather than implementing the full Haskell language at once, our project separates its features into smaller layers, by combining our features, allowing students to slowly be introduced to functional programming.

One important decision is how the syntax/grammar of these features should be organized and formally defined. Specifically we need to figure out how we will allow these features to mesh together in a way that allows people to add and remove them modularly.

This decision matters because the grammar determines which programs are valid, how the language is structured, and how precise our error messages can be. Changing the grammar later could require modifications to the parser, tests, documentation, and existing programs.

If we do not find a consistent approach to defining features and overwrites within them, there will be scalability issues and our compiler will be a nightmare to upkeep.

2. Options

Option 1: Independent Grammars for Each Layer

Each language layer would have its own complete grammar, defining all syntax independently.

This approach would make each layer self contained and easier to understand without referencing other versions. It would also allow layers to develop independently.

However, it would introduce repetition. Shared syntax, such as literals, variables, and arithmetic expressions, would need to be defined multiple times. Any change to a lower level grammar would also need to be applied across its higher counterparts, which kind of defeats the purpose of a layered / feature based langauge.

Option 2: One Unified Grammar

A second option is to define one grammar containing all supported language features, with individual layers restricting which features are available.

This would reduce repetition and allow the project to maintain a single grammar specification. However, it could make the distinction between layers less clear. The parser would also need additional support to reject features that are syntactically valid in the complete language but unsupported in a particular layer.

This approach risks introducing unnecessary complexity into the simpler layers.

Option 3: Incremental Grammar Extensions

The third option is to establish a foundational grammar and define subsequent layers through explicit extensions.

For example, the base grammar defines things that are used in all grammars like variable names, constructor names, digits, literal, etc. Then one of the next layers up, B would define basic types Ints and Bools as well as let expressions. The F layer would then extend on B by introducing function definitions, declarations, and the necessary pattern syntax.

Each layer would preserve the syntax of previous layers while introducing clearly identified additions.

This approach reduces duplication and provides a clearer representation of how the language develops. However, it requires careful documentation of dependencies between layers and rules for resolving potential ambiguities. This is where some sort of overwriting system would need to be implemented, and ifdefs seem like the best option for now, however that comes with its own set of negative side effects.

3. Technical Analysis

The primary considerations that we have encountered so far are maintainability, complexity and ease of understanding the structure of the grammar.

Independent grammars offer flexibility but create duplicated definitions. As the project expands, maintaining consistent syntax across layers becomes increasingly difficult, and defeats the purpose of a feature based grammar.

A unified grammar reduces duplication but shifts complexity into the parser. Each layer must distinguish between syntax that is generally valid and syntax that is permitted in that specific version, this would likely be a pain for features that have conflicting definitions for the same things, like type definitions between B and D.

Incremental extensions seem like a good middle ground by allowing shared definitions while preserving the differences between layers.

For example, a function definition such as:

double x = x + x

should be valid in F but not in B. Under the incremental approach, B defines the expression x + x, while F adds the syntax needed for the declaration double x = x + x

This distinction allows the grammar to communicate which features are introduced at each stage, and which error messages will appear at each stage.

The Haskell 2010 Language Report provides a useful reference for conventional grammar notation and syntax. However, our project will include only the features supported by our simplified language rather than copying the entire Haskell grammar.

4. Recommendation and Verification Plan

I recommend Option 3: Incremental Grammar Extensions.

This approach best supports the project's goal of progressively introducing Haskell features while maintaining an expandable codebase / language.

The initial implementation should establish a formal base grammar then followed by the grammars for B and D and then extensions into E and F. Each grammar production should use consistent notation.

The proposed approach should be verified through several criteria:

Every valid B program must remain syntactically valid in E.

Function declarations must be accepted in F and rejected in B/D/E/None.

Grammar productions must have clear definitions without unintended ambiguity.

Representative valid and invalid programs must produce the expected parsing results, and clear error messages for invalid programs.

New layers / features must be implementable without unnecessarily rewriting existing grammar definitions.

Any layers that must overwrite lower level grammars must be able to do so in a consistent manner.

These criteria would allow our team to determine whether the proposed structure successfully supports language extensibility.

5. Constraints

Our grammar must remain consistent with our project's intended simplified Haskell semantics and any requirements established by the team.

The Haskell 2010 Language Report may be used as a technical reference, but borrowed grammar definitions and notation should be consistent with the Haskell syntax, as the goal is also to be able to compile our language with the Haskell compiler. We should also verify the licensing requirements of any parser libraries / ideas that we use in defining this variable grammar.

Project specifications and implementation details should remain within the github repo and should be clearly communicated through email / team chats.

6. Delegation and Validation

No AI was used in the decisions and drafting of this draft. We have also relegated all use of AI in this project, and thus AI has no place for checks and validations on this project.

7. References

Haskell 2010 Language Report, especially Chapter 2 (Lexical Structure)
https://www.haskell.org/onlinereport/haskell2010/haskellch2.html#x7-140002

Layered Haskell Project Design. Team discussion defining the proposed layers and features.